



A cooperative co-evolutionary genetic algorithm for query recommendation

Debaditya Barman¹ · Ritam Sarkar² · Nirmalya Chowdhury³

Received: 7 March 2022 / Revised: 1 July 2022 / Accepted: 21 April 2023 /

Published online: 29 June 2023

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract

Search engines often recommend a few queries related to the users' original query to help them find the content they are searching. Since extracting the user's intent from the query is a very challenging task due to its short length and ambiguity, it is tough to build a good Query Recommendation system. In this work, we have proposed a Query Recommendation system using a Multi-objective Cooperative Co-evolutionary Genetic Algorithm (QRMOCCEGA) to address the problem. First, we have decomposed the entire problem into two sub-problems. These sub-problems optimise two objective functions, created using users' search behaviours and various string-based similarity methods. QRMOCCEGA uses separate sub-populations to solve the sub-problems simultaneously. It finds complete Pareto-optimal solutions by assembling the relevant members from the two sub-populations which have been collaboratively co-evolved. QRMOCCEGA also maintains diversity in the population. We perform extensive experiments to benchmark our proposed algorithm against several popular algorithms on a large-scale search log extracted from a commercial search engine.

Keywords Query Recommendation · Genetic Algorithm · Multi-objective optimisation · Cooperative co-evolutionary genetic algorithm

Nirmalya Chowdhury contributed equally to this work.

✉ Nirmalya Chowdhury
nirmalya63@gmail.com

Debaditya Barman
debadityabarman@gmail.com

Ritam Sarkar
ritamsrkr.cse@gmail.com

¹ Department of Computer and System Sciences, Visva-Bharati, Santiniketan 731235, West Bengal, India

² Department of Computer Science and Engineering, Indian Institute of Technology Patna, Bihta 801106, Bihar, India

³ Department of Computer Science and Engineering, Jadavpur University, Kolkata 700032, West Bengal, India

1 Introduction

The number of web users reached 5.168 billion in 2021 from only 16 million in 1995¹. Due to this exponential growth, colossal amounts of highly diversified data are generated daily. According to a web server survey by Netcraft² in February 2022, more than 1.17 billion websites existed on the world wide web. Organisation and utilisation of these enormous amounts of information have become critical. Since this information is very diverse and unstructured, mining it to extract some helpful knowledge has become challenging. Search engines were developed to satisfy the information need of Web users. Most search engines have a *keyword*-based interface where users enter different words or phrases to find relevant websites or web pages. It has been found that the length of the user's query is generally very short [29, 54]. Most of the time, these queries are ill-formed, vague, and ambiguous because of their short size. Search engines introduced the query recommendation feature to help users in this regard. According to a survey by CNNIC³ (China Internet Network Information Center), almost 78.2% of users change their original queries to the queries recommended by the search engine if they are not satisfied with the results obtained using their original queries. So **it is essential for search engines to produce high-quality recommendations that can help users get the desired and relevant information more quickly.** Another challenge is to **recommend latent semantically relevant queries concerning the user's original query due to the ambiguities in natural language.**

In recent years, Multi-Objective Genetic Algorithms (MOGA) have been used extensively to solve various applications from different domains. For instance, MOGA has been applied to solve vehicle routing problems [41], radio network optimisation [40], finding overlapping communities [45], job shop scheduling [46] etc. Unfortunately, the performance of the MOGA continuously degrades when the complexity and size of the problem increase. As a result, the set of the non-dominated solutions produced by the MOGA may differ from the actual Pareto front. Moreover, the convergence of the non-dominated solutions returned by the MOGA may also get affected. So to handle a significant complex problem more efficiently, it can be divided into multiple simple sub-problems. These sub-problems can evolve and cooperate to explore the problem's search space more efficiently to find better solutions. In a typical Co-evolutionary system, each sub-problem represents a species, and members from one species can interact with other members from different species [43] to co-evolve. The fitness value of each member has been evaluated based on these interactions. **Co-evolutionary algorithms use several groups (i.e., species) of similar members to find a solution for a multi-objective optimisation problem (MOP).** Typically, Co-evolutionary algorithms can be of two types *Cooperative* [47, 58] or *Competitive* [14, 63] based on whether these species are collaborating or competing with each other.

1.1 Contribution and organisation

This work aims to develop a Cooperative Co-evolutionary Genetic Algorithm to solve the query recommendation problem. Our contributions have been summarised as follows:

- (i) A bipartite graph has been formed using the users' search logs. The depth-limited search technique has been used on this bipartite graph to reduce the search space.

¹ <https://www.internetworldstats.com/stats.htm>

² <https://news.netcraft.com/archives/category/web-server-survey/>

³ <http://research.cnnic.cn/html/1253600840d1370.html>

- (ii) Four features have been constructed to measure relevancy between two queries. Two objective functions have been proposed based on these features.
- (iii) The query recommendation problem has been modelled as a multi-objective optimisation problem.
- (iv) A Cooperative Co-evolutionary GA has been proposed to solve the multi-objective query recommendation problem.
- (v) Performance of our proposed method has been compared with some previously proposed query recommendation techniques and commercial search engines.
- (vi) A ranking technique has been developed to return one solution from the first nondominated front.

The rest of this paper has been organised as follows: Section 2 contains related works. The definition of the problem has been presented in the Section 3, the proposed method has been described in Section 4. Section 5 includes the experimental results, and finally, the concluding remarks have been presented in Section 6.

2 Related works

The objective of a query recommendation system is to provide appropriate queries to the users in response to their original query. Some relevant works related to the query recommendation have been discussed in the following section.

2.1 Query recommendation system

Over the years, researchers used various techniques to develop Query recommendation systems. To analyse and present these techniques comprehensively, we have categorised them into distinct groups based on their similarities using the and reported in the following sections.

2.1.1 Clustering-based techniques

Wen et al. [62] proposed the Density-based Clustering method DBSCAN and its incremental version to cluster queries using user logs. Baeza-Yates et al. [4] found the clusters of semantically similar queries. The search engine's query log has been used to extract the historical records of users' preferences. These preferences have been used in the clustering process. Zahera et al. [67] proposed a query recommendation system based on a clustering technique which forms several groups of semantically alike queries. Anagnostopoulos et al. [2] developed an algorithmic approach to suggest relevant and trendy twitter entities. They first found various clusters of the Twitter entities based on the *virality* and then ranked the queries to generate a recommendation. In [19], authors first find a set of unique query keywords from an input query. Then a relevant set of URLs have been extracted from a URL repository using the set of query keywords. They further extracted different keywords and content words from these URLs. They proposed an Adaptive Pointwise Mutual Information (APMI) algorithm to measure the semantic heterogeneity between the set of query keywords and the content words. A feasible set of words has been created based on the semantic heterogeneity value. Finally, they ranked the URLs based on the semantic similarity value, which was obtained using the APMI algorithm on the set of feasible words and the terms present in the collection of query keywords. Shinde et al. [53] applied a clustering technique to the users' search logs and sessions to form some clusters of queries. They suggested queries based on these clusters.

2.1.2 Similarity measurement-based techniques

Zaïane and Alexander [68] developed a quasi-similarity measurement technique to generate relevant queries. Their proposed approach utilises the number of common URLs between the two set of URLs returned by a search engine for two different queries. Zhang and Nas-raoui [69] modelled users' sequential search behaviour using query logs and combined it with conventional content-based similarity methods to recommend queries. Song et al. [55] proposed a hybrid method to measure semantic similarity between queries. Their proposed method utilises three features: clicked document similarity, associated query similarity, and reverse query similarity. Wang et al. [59] proposed a query recommendation system which can rank similar queries according to their utility. Three utility values have been used in this work: query-level utility, perceived utility, and posterior utility.

2.1.3 Association rule mining-based techniques

Fonseca et al. [23] used association rule mining algorithm to generate a set of related queries to a given query from the user logs of a search engine. Shi and Yang [52] first segmented users' sessions to form query transactions. Then these query transactions have been mined using an improved association rule mining algorithm to extract queries related to an input query. Li et al. [36] developed a parallel FP-Growth algorithm to improve the performance of traditional frequent item set mining algorithms. They used this algorithm to recommend queries. Orthogonal queries are a set of related queries which do not have any syntactic similarity with the input query. Swamy et al. [32] used an association rule mining-based technique to generate a set of orthogonal queries. They used confidence and diversity to rank the rules. Few other orthogonal query recommendation techniques have been proposed in [58, 64].

2.1.4 Language model-based techniques

He et al. [27] examined different query sequence models (e.g. Naive variable-length N-gram model, Variable Memory Markov (VMM) model, and Mixture Variable Memory Markov (MVMM) model). They showed that these algorithms perform better than the conventional pair-wise approaches. Finally, they used MVMM to build their query recommendation system and extensively tested their algorithm on two large-scale query logs. Liu et al. [35] developed two snippet click behaviour models from the large-scale users' search behaviour. These two models are- the *global scale snippet click model* and the *local scale snippet click model*. In *global scale snippet click model*, all the clicked snippets for a given query have been treated as a single snippet document. But in the *local scale snippet click model*, these snippets have been considered separate documents. They developed two different query recommendation systems based on these two models and observed that the higher-ranked queries for these two systems are very similar. Zhao et al. [70] used tensor representation to model the relation between the users, users' locations, and users' queries. Now, the proposed system learned the tensor function to predict the search intent of a user. Hanauer et al. [26] used *MetaMap* technique to extract medical contexts from search queries and used it to develop a query recommendation system for the healthcare domain. Ghali and Qadi [21] extracted query contexts using a language model in 2017. These contexts and LSA (Latent Semantic Analysis) method have been used to develop the recommendation system. Duan et al. [20] developed a system which uses the topic modelling technique to understand the true meaning of a user's

query and then recommends relevant queries. These recommended queries have semantic relations with the original query.

2.1.5 Graph mining-based techniques

Cheng et al. [13] proposed a method based on random walk technique to generate query recommendations from a k-partite graph. Their proposed method can handle various heterogeneous features. Anagnostopoulos et al. [3] built a query-flow graph from the query logs. Here query recommendation problem has been considered as a problem of placing some shortcuts in the said graph such that the expected utility value can be maximised during a random walk. Ma et al. [38] considered each query as a node in a graph. Then they proposed a heat diffusion method to propagate similarities between different nodes and generate a set of queries for recommendation. Bonchi et al. [8] used the concept of a centre-piece subgraph to recommend *long-tail* (i.e. rare) queries. Ahmedi and Shabani [1] used a social network analysis-based ranking algorithm to recommend queries to the users. At first, Huang et al. [28] explored several entities from queries. Then these entities have been used to suggest new queries by consulting with multiple information sources.

2.1.6 Other techniques

Tobías et al. [22] used a variation of collaborative filtering, which uses a query log to recommend queries to the users. Qiao et al. [49] mined query logs to identify competitive keywords from search logs to improve advertising strategies of search engines. Guo et al. [25] proposed a *Query Utility Model* which can extract utility values of the queries from users' search sessions. Their system can recommend queries with high utility values. Ma and Zhang [37] proposed a machine learning-based query recommendation system which can support *exploratory search*. Search goal often changes in this *exploratory search* technique. Li et al. [34] proposed a system to capture users' search intention based on users' sequential search behaviour and feedback on previously suggested queries. Senthilkumar and Ch [51] developed a domain knowledge-based search engine. A collaborative search process has been used here to improve the performance of this search engine. Tekli et al. [56] proposed a semantic-aware inverted index structure to generate semantically relevant queries. They used a collection of textual data and a semantic knowledge base to build the index. In the next year, they [57] developed another semantic aware indexing technique to suggest semantically relevant queries. They used textual as well as semantic information for this process. They also proposed a parallel query processing algorithm to speed up the process. The recommendation system proposed by Câmara and Santos [10] traverses a query log which is semantically annotated to suggest queries.

Generally, a query recommendation system recommends similar queries in response to the users' original query. The similarity between two queries can be measured using various features. Since the goal of a query recommendation system is to recommend only few queries with the highest similarity score, the query recommendation problem can be modelled as an optimisation problem. Very few above-mentioned research works [3, 38] treated query recommendation problem as an optimisation problem to develop a solution. Moreover, multiple similarity measurement techniques can be defined to measure different kinds of similarities between the queries. Several objectives can be formed using these techniques. The use of multiple objectives can make the recommendation system more robust as well as useful. So, we have treated this problem as a multi-objective optimisation problem and developed

a GA-based solution. To the best of our knowledge, very few research works [6, 55] in the domain of query recommendation systems used GA. Song et al. [55] used single objective GA to cluster the queries. They measured efficacy of different techniques to measure similarity between two queries. Moreover, during similarity calculation they used web snippets which may not be available in typical search logs. Barman et al. [6] also used Single objective GA to develop a Query Recommendation system. Unlike [6, 55], our proposed Query Recommendation system uses two objective functions and these functions have been created using four different decision variables. First objective function utilises two decision variables and it helps us to capture users' search behaviors. The other two decision variables capture visual similarities between two queries. Second objective function uses these two variables. So, these objective functions represent two very different aspects of Query recommendation and we have been able to generate high quality recommendations by finding the Pareto optimal solutions. Moreover, our proposed approach returns the Pareto front which may contain a set of Pareto optimal solutions. This set may be used to generate *personalized* recommendations. We have used multiple GAs to optimise the objectives simultaneously. These GAs cooperate with each other in order to build the final solution. Our approach improves running time as well as quality of the solutions. Detailed discussion on this can be found in Section 4.3.

In the next section, we have discussed some relevant works on multi-objective optimisation.

2.2 Multi-objective optimisation-based recommendation system

In *multi-objective optimisation*, a set of objective functions get optimised simultaneously. Researchers extensively used NSGA-II [18] and NSGA-III [17] algorithms to solve the various multi-objective optimisation problems.

2.2.1 Multi-objective GA-based techniques

Accuracy and diversity are the two most common evaluation metrics for a traditional recommendation system. Unfortunately, conflict may arise between these two metrics. Zuo et al. [72] optimised these two performance metrics using the NSGA-II-[18] based multi-objective evolutionary algorithm (MOEA) for their recommendation system. They applied the clustering technique to form groups of similar users to improve computational efficiency. Items with very few ratings are called *long-tail* items. Wang et al. [60] used two objective functions to generate popular and *long tail* items. Their proposed MOGA optimises these two conflicting objective functions. Cui et al. [16] proposed the NSGA-II [18] based MOEA to optimise the precision and diversity of the recommendations. They used a probabilistic multi-parent crossover operator to prioritise the items with high frequency. In [31], authors used MOGA to implement memory based-collaborative filtering (CF) system. Their proposed system selects a group of users relevant to an active user in such a way that these users should be well-diversified and similar to the active user.

2.2.2 Other multi-objective optimisation techniques

Chai et al. [11] proposed a hybrid method based on Singular Value Decomposition (SVD) and multi-objective immune algorithm (MOIA). SVD has been used to decompose the rating matrix, which is very sparse. Geng et al. [24] proposed a two-phase hybrid location recommendation system. In the first phase, they used CF to generate candidate items for the

recommendation. They also improved this CF by using information from the social network of an active user. In the second phase, they used MOIA to optimise the similarity between the historic check-in behaviours of the users and geographical influence. Cai et al. [9] proposed a hybrid many-objective Reference Vector Guided Evolutionary Algorithm (RVEA) [12] to improve the recommendations' accuracy, diversity, coverage, and novelty. They compared the performance of the proposed algorithm with the other three algorithms. These are NSGA-III [17], Grid-based Evolutionary Algorithm (GrEA) [66], and Strength Pareto EA 2 with Shift-based Density Estimation (SPEA2 + SDE) [33]. The running time of RVEA has been improved by incorporating the k-means clustering algorithm, which forms several groups of users based on multiple attributes. The query recommendation system is a bit different from than traditional item recommendation system. Most of the above-mentioned item recommendation systems operate on the user-item rating matrix. Generally, each cell of this matrix holds a value of 0 to 5 or 0 to 10, which denotes the users' satisfaction level. The similarity between two users can be computed based on their ratings. Sometimes, users' profiles can also be used to calculate the similarity value. These similarity values can then be used to improve the accuracy and performance of the traditional item recommendation system. For instance, different clustering techniques [9, 72] have been used to find groups of similar users. In the case of the Query Recommendation system, recommendations are generated based on the logs generated during the search sessions. A search log consists of session id, user's query, date, time, URL visited for the query, and rank of the URL. The user's profile data is not always available for security reasons. Furthermore, it is impossible to form groups of similar users by just looking at the logs. So, to generate a list of relevant recommendations, we have extracted the features from the search log itself. The Query recommendation problem has been modelled as a MOO using two objective functions constructed using these extracted features. A detailed discussion of the feature extraction techniques and construction of objective functions can be found in the Section 4.

3 Query Recommendation problem

Let a set of n number of queries is Q . For simplicity, Q is assumed to be finite and non-empty. For a given query $q_0 \in Q$, the system should recommend k number of relevant queries $Q' = \{q_r, q_s, \dots, q_t\}$ and $|Q'| = k$ from Q , $Q' \subseteq Q$ in such a way that it satisfies following conditions.

- 1 $q_0 \notin Q'$
- 2 All the queries of Q' should be distinct. (i.e. q_r, q_s , and q_t should be different from each other.)
- 3 All the queries of Q' should be similar or relevant to q_0 (i.e. q_r, q_s, q_t should be similar or relevant to q_0 .)
- 4 All the queries of Q' should be similar or relevant to each other (i.e. q_r should be similar to q_s, q_s should be similar to q_t and q_t should be similar to q_r .)

Let $C_1, C_2, \dots, C_l$ denote the constraints or conditions (e.g. conditions i, ii etc.) imposed over Q to select Q' . C_i can be represented as following predicate.

$C_i: 2^Q \rightarrow \{T, F\}$ where T and F represent two binary values *True* and *False*, respectively and $i \in \{1, 2, \dots, l\}$.

Let there exists x number of relevance measurement techniques. The optimisation criteria of these techniques are represented by $f_1, f_2, \dots, f_x$. Assume F is the set of all the optimisation criteria or objective functions and all the objective functions are minimization in

nature. $F = \{f_1, f_2, \dots, f_x\}$. So, a multi-objective function (i.e. minimization) can be often defined as follows.

$$F : Q' \rightarrow \mathbb{R}^x \quad (1)$$

So the basic mathematical formulation of our problem is as follows. Find $\tilde{Q}, \tilde{Q} = \{Q'_1, Q'_2, \dots, Q'_m\}$ and $Q'_i \in Q$ such that $C_i(Q'_i) = \{True\}$ for $i = 1, 2, \dots, l$ and each Q'_i should be Pareto optimal. A Q'_i is said to be Pareto optimal when it does not dominated by any other feasible Q'_j .

A feasible solution Q'_s is said to dominate another solution Q'_t , if the following two conditions satisfy.

- 1 $f_i(Q'_s) \leq f_i(Q'_t) \forall i \in \{1, 2, \dots, x\}$
- 2 $f_i(Q'_s) < f_i(Q'_t) \exists i \in \{1, 2, \dots, x\}$

Weighted sum of all the optimisation functions could be used to form a single optimisation function which is minimization in nature. Let the single optimisation function is f_w . Then we need to find Q' such that $Q' \subseteq Q$ and $C_i(Q') = \{True\}$ for $i = 1, 2, \dots, l$ and $f_w(Q') \leq f_w(P)$ for any feasible $P, P \subseteq Q$.

Above stated problem can easily be reduced to the well-known *Optimal Subset Selection* (OpSS) problem. OpSS finds a subset of a given set based on constraints and an optimisation criterion (maximum or minimum). Since OpSS is an NP-hard problem, our problem can be claimed as NP-Hard [7, 48, 61].

4 Proposed method

A user's search session is usually stored as the sextuple depicted in Fig. 1 in a log maintained by the search engine.

The most important component of this sextupled server log is the query (q_i) and clicked URL link (l_j) pair. This information can be presented by an undirected bipartite graph $G = (U, V, E)$ where, $U = \{q_0, q_2, \dots, q_{n-1}\}$, $V = \{l_0, l_2, \dots, l_{p-1}\}$ and if the link l_j is clicked for the query q_i then $E = \{(q_i, l_j)\}, \forall i, \forall j$. It may be noted that this bipartite graph has been constructed only once and used for different root queries. Now, due to the huge number of entries in the search log, the size of this bipartite graph will be huge. So a sub-graph has been extracted from this bipartite graph based on Depth Limited Search (DLS) from a given root query (q_0). Please note that the *depth limit* has been set to two here. The lower value of the *depth limit* will reduce the computational time. Figures 2 and 3 present the process of estimating the relevancy between two queries. The edge weight of the bipartite graph in Fig. 2 specifies how many times a link (l_j) has been clicked for a query (q_i).

We can define the probability of a query and the occurrence of a link together as follows.

$$p(q_i, l_j) = \frac{n_j}{\sum_{k=1}^r n_k} \quad (2)$$

where n_j is the number of times a link l_j is clicked for a query q_i , and r is the number of distinct links clicked for the query q_i .

Session id	Query	Date	Time	Rank of the clicked URL link	Clicked URL link
------------	-------	------	------	------------------------------	------------------

Fig. 1 Search session of a user

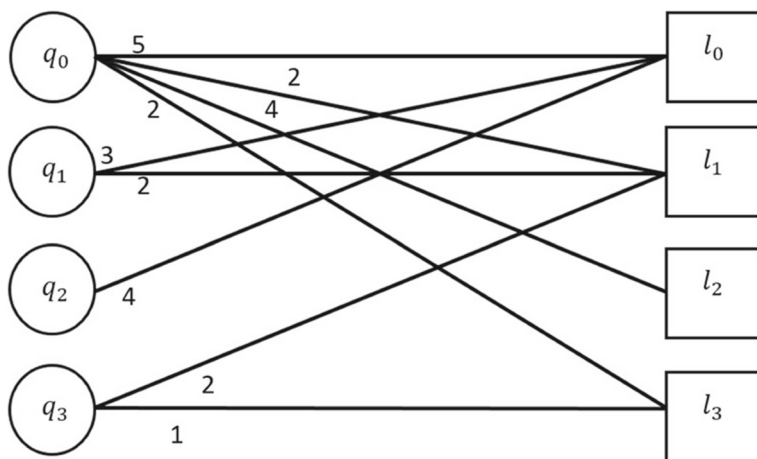


Fig. 2 Query-Link bipartite graph

According to the Fig. 2, $p(q_0, l_0) = \frac{5}{13}$, $p(q_0, l_1) = \frac{2}{13}$, $p(q_0, l_2) = \frac{4}{13}$, $p(q_0, l_3) = \frac{2}{13}$, $p(q_1, l_0) = \frac{3}{5}$, and $p(q_1, l_1) = \frac{2}{5}$.

The relevancy between two queries can be defined using the commonly clicked links. Let m is the total number of such links.

$$cp(q_i, q_k) = \sum_{j=1}^m p(q_i, l_j) \times p(q_k, l_j) \quad (3)$$

where the link l_j has been clicked for both the queries q_i and q_k .

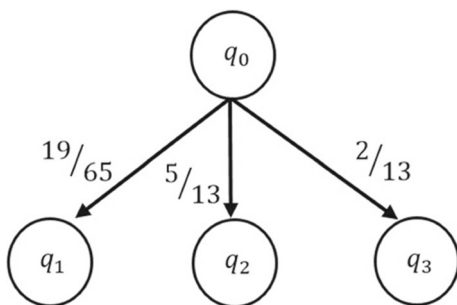
According to the Fig. 1, common links clicked for the queries q_0 and q_1 are l_0 and l_1 .

$$cp(q_0, q_1) = p(q_0, l_0) \times p(q_1, l_0) + p(q_0, l_1) \times p(q_1, l_1) = \frac{19}{65} \quad (4)$$

In this way, we can generate the directed graph shown in Fig. 3. Similarly, we may form the graph shown in Fig. 4 for n number of relevant queries. Let's call the Graph $G_{\hat{Q}}$.

To generate the relevant query subset (i.e. $\hat{Q}$) we have only considered the queries those are within one hop from a given root query q_0 , $\hat{Q} = \{q_1, q_2, \dots, q_{\hat{n}}\}$ and $|\hat{Q}| = \hat{n}$. Since $\hat{n} \ll n$, where, n is the number of all the distinct queries ($|Q| = n$), the search space has been drastically reduced.

Fig. 3 Relevance of queries w.r.t. root query.



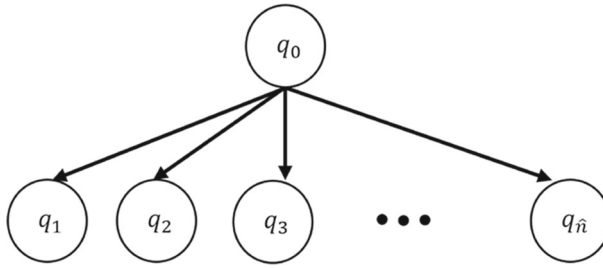


Fig. 4 Relevant queries

The relevancy between two queries has been measured using the following four features.

1 Compound probability (cp):

Compound probability measures the degree of relevancy between two given queries. Assume a link l_t is clicked for two queries q_r and q_s . $p(q_r, l_t)$ denotes the probability of clicking the link l_t when the query is q_r and $p(q_r, l_t) = 0.4$. Similarly, $p(q_s, l_t) = 0.5$. According to (3), $cp(q_r, q_s) = 0.02$. A higher value of cp indicates a higher degree of relevancy.

2 Jaccard index of clicked links (ji_l):

Different users can click multiple links for the same query. If same links have been clicked for two different queries, then these two queries should be relevant to each other. Suppose $ji_l(q_r, q_s)$ denotes the degree of relevancy between the queries q_r and q_s when the set of links L_{q_r} and L_{q_s} are clicked for q_r and q_s respectively, have been provided.

$$ji_l(q_r, q_s) = \frac{|L_{q_r} \cap L_{q_s}|}{|L_{q_r} \cup L_{q_s}|} \quad (5)$$

Suppose $L_{q_r} = \{l_1, l_3, l_5, l_6, l_9\}$, $L_{q_s} = \{l_1, l_2, l_5, l_6, l_7\}$, and $L_{q_t} = \{l_1, l_6, l_8, l_9\}$. According to (5), $ji_l(q_r, q_s) = \frac{3}{7}$ and $ji_l(q_r, q_t) = \frac{1}{2}$. So q_t is more relevant to q_r than q_s to q_r .

3 Jaccard index of words (ji_w):

A query can be formed using multiple words. So a set of unique words can be used to represent a query. For instance, a query $q_r = \text{best family movies}$ can be represented as a set of words W_{q_r} , $W_{q_r} = \{\text{best, family, movies}\}$. If the exact words appeared in two different queries, then these two queries should be similar to each other. $ji_w(q_r, q_s)$ determines the level of similarity between two queries q_r and q_s .

$$ji_w(q_r, q_s) = \frac{|W_{q_r} \cap W_{q_s}|}{|W_{q_r} \cup W_{q_s}|} \quad (6)$$

Assume $W_{q_r} = \{w_1, w_2, w_5, w_7\}$, $W_{q_s} = \{w_1, w_2, w_3, w_5\}$, and $W_{q_t} = \{w_2, w_5\}$. So, according to the (6), $ji_w(q_r, q_s) = \frac{3}{5}$ and $ji_w(q_r, q_t) = \frac{1}{2}$. So q_s is more similar to q_r than q_t to q_r .

4 Ratio of the Length of Longest Common subsequence to the maximum length of the queries (lcs_{ratio}):

A query can be represented as a string. The longest subsequence which can be found in two queries is called the Longest Common Subsequence (LCS). Ratio of the length of LCS of

two queries q_r and q_s and the maximum length of the queries q_r and q_s is $lcs_{ratio}(q_r, q_s)$.

$$lcs_{ratio}(q_r, q_s) = \frac{\text{len}(LCS(q_r, q_s))}{\text{max_len}(q_r, q_s)} \quad (7)$$

Suppose $q_r = \text{computer science}$, $q_s = \text{computer science jobs}$, $q_t = \text{computer price}$. So according to the (7), $lcs_{ratio}(q_r, q_s) = \frac{16}{21}$ and $lcs_{ratio}(q_r, q_t) = \frac{9}{16}$. So q_s is more similar to q_r than q_t to q_r .

Search engine often recommends some queries which look similar to the users' original query. For example, a search engine may recommend queries like *computer price*, and *computer table* in response to the original query *computer*. But latent semantically relevant queries like *processor* and *laptop* are challenging to generate. Among the above-stated four features, first, two features cp and ji_l have been used to generate latent semantically relevant queries, and the last two features ji_w , and lcs_{ratio} have been used to generate queries that look similar.

Algorithm 1 presents the mechanism to generate the relevant query subset (i.e. $\hat{Q}$).

Algorithm 1 Relevant query subset generation

Ensure: q_0 is the root query; $qlinfo$ is a list of lists where each list contains q_0 , $Clink$, and $clknum$.

Ensure: $\hat{Q}$ contains the set of all unique relevant queries; $cprob$; LCS ; JI_{word} ; JI_{link} .

```

1: if  $qlinfo == NULL$  then
2:   Query is not present in the dataset
3: else
4:   Initialize  $num_{link}$  = number of unique links for  $q_0$ ,  $total_{click}$  = total number of clicks for  $q_0$ ,  $P_{rtol} = []$ ,
    $P_{rtog} = []$ ,  $ltoq = []$ ,  $P_{ltoq} = []$ ,  $\hat{Q} = []$ ,  $cprob = []$ ,  $LCS = []$ ,  $JI_{link} = []$ ,  $JI_{word} = []$ 
5:   for  $i \leftarrow 0$  to  $num_{link}$  do
6:      $P_{rtol}[i] = \frac{clknum[i]}{total_{click}}$ 
7:   end for
8:   for  $i \leftarrow 0$  to  $num_{link}$  do
9:      $ltoq[i] \leftarrow$  the queries except the root query for which the link  $l_i$  has been clicked.
10:    for  $j \leftarrow 0$  to  $|ltoq[i]|$  do
11:      Calculate  $P_{ltoq}[j]$ 
12:       $P_{rtog}[i][j] \leftarrow P_{rtol}[i] \times P_{ltoq}[j]$ 
13:    end for
14:  end for
15:   $\hat{Q} \leftarrow \text{Remove Duplicate}(ltoq)$ 
16:  for  $i \leftarrow 0$  to  $|\hat{Q}|$  do
17:    for  $j \leftarrow 0$  to  $|ltoq[j]|$  do
18:      if  $\hat{Q}[i]$  is in  $ltoq[j]$  then
19:         $pos \leftarrow$  position of  $\hat{Q}[i]$  in  $ltoq[j]$ 
20:         $cprob[i] = cprob[i] + P_{rtog}[j][pos]$ 
21:      end if
22:    end for
23:  end for
24:  for  $i \leftarrow 0$  to  $|\hat{Q}|$  do
25:     $LCS[i] = lcs_{ratio}(q_0, \hat{Q}[i])$ 
26:     $JI_{word}[i] = ji_w(q_0, \hat{Q}[i])$ 
27:     $JI_{link}[i] = ji_l(q_0, \hat{Q}[i])$ 
28:  end for
29:  Return  $\hat{Q}$ ,  $cprob$ ,  $LCS$ ,  $JI_{word}$ ,  $JI_{link}$ 
30: end if

```

4.1 Description of algorithm 1

Algorithm 1 takes q_0 , and $qlinfo$ as input where q_0 is the root query and $qlinfo$ is a list of lists containing q_0 , $Clink$, and $clknun$. $Clink$ is the set of links clicked for q_0 , and the $clknun$ implies the number of times a link in $Clink$ has been clicked for q_0 .

Algorithm 1 generates $\hat{Q}$, $cprob$, LCS , JI_{word} , and JI_{link} as output. $\hat{Q}$ contains the set of all unique, relevant queries, whereas $cprob$, LCS , JI_{word} , and JI_{link} are the compound probability, LCS ratio, Jaccard Index of words, and Jaccard Index of links between q_0 and each of the queries in $\hat{Q}$, respectively.

Line 1 checks whether the q_0 exists in the dataset or not. In line 4, num_{link} , $total_{click}$, P_{rtol} , P_{rtolq} , $ltoq$, P_{ltoq} , $\hat{Q}$, $cprob$, JI_{link} , and JI_{word} have been initialized. The total number of unique links those have been clicked for q_0 is represented as num_{link} , $total_{click}$ is the total number of times the links are clicked for q_0 , P_{rtol} contains the probabilities of the links to be clicked for a root query q_0 . All the queries except the root query for which a link in $Clink$ has been clicked have been stored in $ltoq$. The link to query probability for each of the queries in $ltoq$ has been kept in P_{ltoq} . All the distinct queries of $ltoq$ have been stored in $\hat{Q}$. The compound probability of the relevancy between q_0 and each of the queries in $ltoq$ has been stored in P_{rtolq} . The $cprob$ contains new compound probabilities by taking the summation of compound probabilities from P_{rtolq} , if any query shares multiple common links with the q_0 . The LCS , JI_{word} , and JI_{link} contain the LCS ratio, Jaccard index of words, and Jaccard index of links, respectively, between the root query and each of the queries in $\hat{Q}$. Line 5 to 7 calculate P_{rtol} (i.e. the probability of clicking a particular link for q_0). It has been calculated for each of the links clicked while searching for the q_0 . Let's consider a link l_i has been clicked for the root query q_0 . Then in line 9, all the other queries (except the root query q_0) for which the link l_i has been clicked have been discovered and stored in $ltoq$. Therefore, $ltoq$ may contain several duplicate queries, as multiple links can be clicked for a single query. So, it is necessary to compute the P_{rtolq} for all the queries in $ltoq$ separately in lines 10 to 13 and summed together later in lines 17 to 22 during the calculation of $cprob$, in case multiple links have been clicked for a single query. All the distinct links of $ltoq$ are stored in $\hat{Q}$ in line 15. From line 24 to 28 LCS , $jaccard_index_word$, and $jaccard_index_link$ have been calculated by invoking the functions lcs_ratio , ji_w , and ji_l , respectively for each of the queries of $\hat{Q}$. Finally, the outputs have been returned in line 29.

4.2 Description of the objective functions

Let's assume that the root query is q_0 , and we need to recommend k number of queries (i.e. Q' , $|Q'| = k$). Two objective functions have been used here. The first function defined in (8) ensures that the recommended queries are latent semantically relevant to the root query and also to each other. The second objective function defined in (9) is responsible for recommending queries which look similar to the users. Following the multi-objective optimisation problem represents the query recommendation problem.

Given a set of queries (i.e. $\hat{Q}$) relevant to a root query (q_0), we need to choose k number of queries in such a way that the following two functions, f_1 and f_2 are optimised.

$$\text{minimize } f_1(\hat{Q}) = \sum_{i=1}^k \| LSem(q_0) - LSem(q_i) \|$$

$$+0.5 * \sum_{i=1}^k \sum_{j=1}^k \| L\text{Sem}(q_i) - L\text{Sem}(q_j) \| \quad (8)$$

where,

$$L\text{Sem}(q_i) = (cp(q_0, q_i), jil(q_0, q_i))$$

$$\begin{aligned} \text{minimize } f_2(\hat{Q}) = & \sum_{i=1}^k \| \text{Sim}(q_0) - \text{Sim}(q_i) \| \\ & +0.5 * \sum_{i=1}^k \sum_{j=1}^k \| \text{Sim}(q_i) - \text{Sim}(q_j) \| \end{aligned} \quad (9)$$

where,

$$\text{Sim}(q_i) = (ji_w(q_0, q_i), lcs_{ratio}(q_0, q_i))$$

subject to

$$q_0 \notin Q' \quad (10)$$

$$\exists q_i \in Q' : (P(q_i) \wedge \forall q_j \in Q' : P(q_j) \rightarrow q_i = q_j) \quad (11)$$

The first objective function presented in (8) minimizes the cost to ensure that all the recommended queries are latent semantically relevant to the root query and also to each other. The second objective function presented in (9) minimizes the cost to ensure that all the recommended queries are visually similar to the root query and also to each other.

Constraints presented in (10) and (11) ensure that the root query is not present in the set of recommended queries and that all the recommended queries are unique, respectively.

4.3 A Query Recommendation system using Multi-objective Cooperative Co-evolutionary Genetic Algorithm (QRMOCCGA)

Potter and De Jong [47] introduced the concept of a Cooperative Co-evolutionary algorithm (CCEA) in 1994. This approach is based on the *divide and conquer* paradigm. It divides a large problem into smaller sub-problems and then solves each sub-problem using different Evolutionary Algorithms (EA). These EAs work concurrently and independently. Thereby, to generate the solution for a large problem, the solution of each EA cooperates among themselves. How well a CCEA can solve an optimisation problem depends on the *separability* property of the problem.

Definition 1 An optimisation problem $g(X) : \omega \rightarrow \mathbb{R}$ is fully separable [65] if it can be solved by optimizing each decision variable separately.

$$\arg \max_{x_1, \dots, x_n} g(X) = \left(\arg \max_{x_1} g(x_1, \dots), \dots, \arg \max_{x_n} g(\dots, x_n) \right) \quad (12)$$

where ω is an n -dimensional decision space.

Definition 2 An optimisation problem $g(X)$ is fully non-separable [39] if each possible pair of decision variables are dependent with each other.

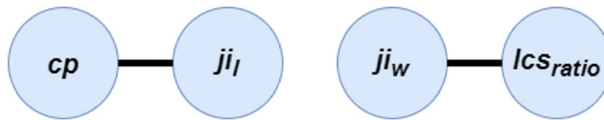


Fig. 5 Decision variables

Definition 3 An optimisation problem $g(X)$ is k non-separable [15] if at most k number of decision variables are interdependent.

In our optimisation problem, we have four decision variables (i.e. cp , ji_l , ji_w , lcs_{ratio}). According to the nature of these four decision variables, we can say that the optimisation problem we are dealing with is a 2-non separable problems with two independent sub-components, as shown in Fig. 5. These two sub-components represent the two objective functions f_1 and f_2 defined in the (8) and (9), respectively. Since CCEA can easily solve fully separable as well as k non-separable with lesser k values, we have used the Cooperative Co-evolutionary multi-objective approach to develop our proposed method.

As our problem consists of two objective functions, two different Genetic Algorithms (GA) have been used to optimise these functions. These two GAs act as slave GAs. The first slave GA optimises the function f_1 and the second slave GA optimises the function f_2 . The solutions generated by these slave GAs satisfy the constraints presented in (10) and (11).

Each slave GA has its own sub-population. A Master GA uses the sub-populations generated by the slave GAs in every iteration. The architecture of the proposed QRMOCGA having two objectives has been presented in the Fig. 6. We need to use n number of slave GAs if an optimisation problem has n number of objectives. Master GA uses a cooperation phase where the solutions generated by individual GA (i.e. solutions of the sub-problems) have been merged and evaluated. Like Slave GAs, Master GA also applies the genetic operators (i.e. crossover, mutation) on these populations. Overview of the whole procedure can be found in the Fig. 7.

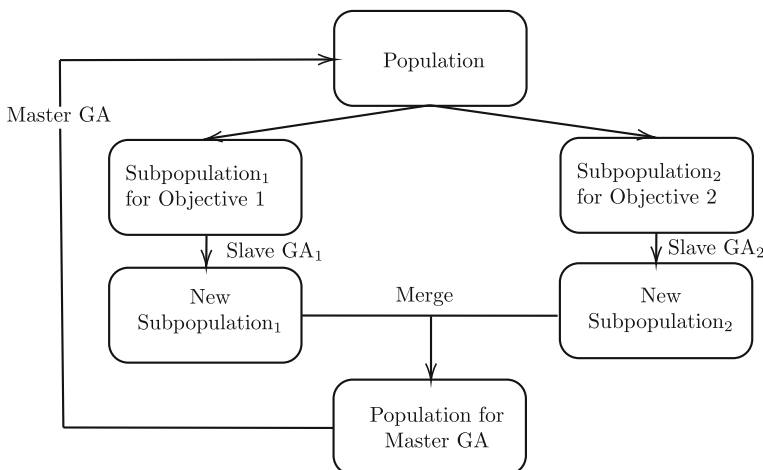


Fig. 6 Abstract view of the proposed QRMOCGA

4.3.1 Slave GA

One Slave GA works on only one objective function. Slave GAs are responsible for independently optimising their concerning objectives to get the optimal solutions for each of the individual goals. The steps of a Slave GA have been described below.

1 Representation of the chromosome:

A chromosome has been represented by a string of length k where the k is the number of recommended queries. Each character of this string (i.e. gene) represents a query relevant to the root query (i.e. q_0). The Fig. 8 illustrates the design scheme of the chromosome. Every chromosome represents a solution to the query recommendation problem.

2 Initial population:

Each chromosome in the initial population (i.e. X , $pop_{init} = |X|$) has been created by taking k number of queries from $\hat{Q}$, $|\hat{Q}| = \hat{n}$. All the queries in a chromosome should satisfy the constraints presented in (10) and (11). If $\hat{n} \bmod k = y$ and $y \neq 0$ then $k - y$ number of queries have been chosen randomly from $\hat{Q}$ in such a way that the above

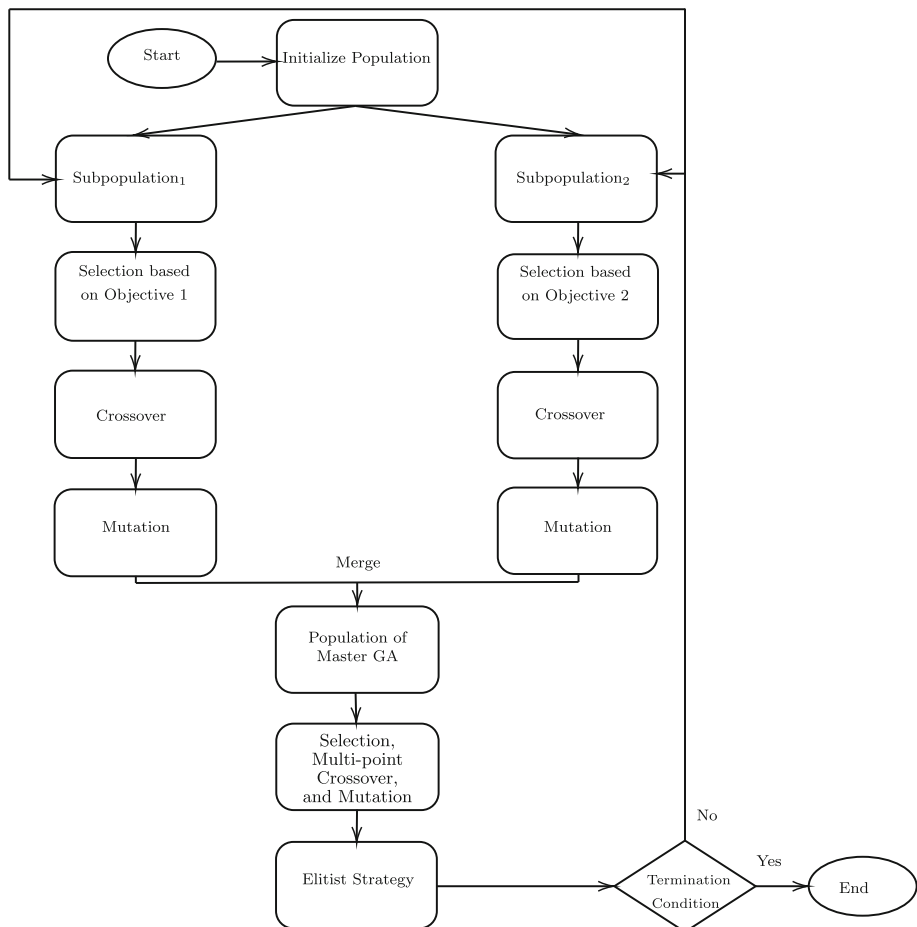


Fig. 7 The architecture of the QRMOCGA

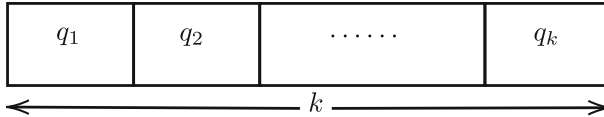


Fig. 8 Representation of the chromosome

mentioned constraints are satisfied. So, the population size (i.e. $pop_{init} = \lceil \frac{n}{k} \rceil$) varies from a root query to another root query. Since we have two objectives, two slave GAs have been used in the proposed QRMOCGA model. Each slave GA (i.e. $slaveGA_i$) has its own sub-population (i.e. $Subpop_i$). Hence, the entire population (i.e. $|X|$) has been partitioned into two sub-populations (i.e. $Subpop_1, Subpop_2$) randomly.

3 Chromosome evaluation:

A chromosome ($x_{1i}, x_{1i} \in Subpop_1$) in $Subpop_1$ has been evaluated using the first objective function described in (8). Similarly, a chromosome ($x_{2i}, x_{2i} \in Subpop_2$) in $Subpop_2$ has been evaluated using the second objective function described in (9).

4 Genetic Operators:

There are three genetic operators. These are Selection, Crossover, and Mutation.

- **Selection:** We have defined a selection probability ($P(x_{hi}), x_{hi} \in Subpop_h$) for each chromosome (x_{hi}) in a sub-population (i.e. $Subpop_h, Subpop_h \equiv X_h$) of $h - th$ slave GA to be selected as a parent. The objective function of the $h - th$ slave chromosome is represented by f_h .

$$P(x_{hi}) = \frac{f_h(x_{hi}) - f_{min_h}(X_h)}{\lambda_h * (f_{max_h}(X_h) - f_{min_h}(X_h))} \quad (13)$$

where,

$$\begin{aligned} \lambda_h &= |X_h|, \\ f_{min_h}(X_h) &= \min\{f_h(x_{hi}) | x_{hi} \in X_h\}, \text{ and} \\ f_{max_h}(X_h) &= \max\{f_h(x_{hi}) | x_{hi} \in X_h\}. \end{aligned}$$

- **Crossover:** A variable crossover probability (P_{co}) has been defined for each slave GA to decide whether to perform a crossover operation or not. This probability decreases with time or iteration. If the crossover operation did not get performed then offsprings or children are clones of the parent. We have randomly generated at most ($\eta = \lceil \frac{k}{3} \rceil$) number of crossover points such that the children satisfy the constraints defined in (10) and (11).
- **Mutation:** A fixed mutation probability ($P_{mu}, P_{mu} \ll P_{co}$) has been defined for each slave GA to decide whether to perform a mutation operation on the child chromosome or not. Another probability P_{mu_ge} ($P_{mu_ge} = \frac{1}{k^2}$, where k is the length of the chromosome) has been defined to decide whether to perform mutation to a gene of the chromosome or not if a child chromosome has been selected for mutation operation. Each gene in a chromosome represents a query. If a gene has been selected for mutation operation, then it has been replaced by a randomly selected query from $\hat{Q}$ such that it satisfies all the constraints.

4.3.2 Master GA

The objective of the Master GA is to combine the solutions generated by the Slave GAs (i.e. *cooperation* phase) and conduct a searching operation along the Pareto front of all the objective functions with the hope of getting optimal global solutions. The chromosome representation scheme, crossover operation, and mutation operation of the Master GA have been considered identical to the slave GA. Only the initial population, selection operation, and chromosome evaluation steps are different. Moreover, there is an additional step in Master GA named elitist strategy.

1 Initial Population:

The initial population for Master GA has been generated by combining the solutions of Slave GAs and deleting the duplicate chromosomes.

2 Chromosome Evaluation:

In Master GA, we have used two different chromosome evaluation techniques. These are a. Non-dominated sorting based chromosome evaluation [18], b. k-th nearest neighbour estimator-based chromosome evaluation [71].

- *Non-dominated sorting based chromosome evaluation:* This chromosome evaluation technique has been used in selection operation to assign fitness values to the chromosomes according to their goodness. For this purpose, we have performed non-dominated sorting on the population of chromosomes which provides $\hat{p}$ number of Pareto fronts. Therefore, the chromosomes belonging to the first non-dominated front have been assigned with a fitness value of $\hat{p}$; similarly, the chromosomes belonging to the second non-dominated front have been assigned with a fitness value ($\hat{p} - 1$) and so on.
- *k-th nearest neighbour estimator based chromosome evaluation:* This chromosome evaluation technique has been used to develop an *elitist* strategy. The inverse distance between a chromosome and its k-th nearest neighbour indicates the density of that particular chromosome. When the population size of Master GA exceeds a predefined size, this operation has been used to reduce the population size by pruning highly crowded chromosomes. It also helps to preserve the diversity in the population.

The algorithm for chromosome pruning technique based on k-th nearest neighbour (i.e. CPKNN()) has been presented in the Algorithm 2. It takes $Size_{pop}$ and Pop_{Master} as input. It returns Pop_{Master} whose size is equal to $Size_{pop}$, after pruning the chromosomes in the dense region. $Size_{pop}$ is the predefined population size, and Pop_{Master} is the population of Master GA after merging the current population with the off-springs. In line 1, the value of k has been considered as $\lceil \sqrt{Size_{pop}} \rceil$. In line 7, two is added with $distance_{ij}$ in the denominator of $density_{ij}$ to ensure that the value of $density_{ij}$ lies in the interval (0,1).

3 Selection:

In the selection operation, fitness value are first assigned to each chromosome of the Master GA based on their rank obtained through non-dominated sorting. After that, the following selection operator (i.e. the selection operator of slave GA) has been used to select the parent chromosomes.

$$P(x) = \frac{fit_{nm}(x) - fit_{nm_{min}}(X)}{\lambda * (fit_{nm_{max}}(X) - fit_{nm_{min}}(X))} \quad (14)$$

where X is the population of Master GA, $\lambda = |X|$, $fit_{nm}()$ returns the fitness value of a chromosome based on non-dominated sorting, $fit_{nm_{min}}(X) = \min\{fit_{nm}(x) | x \in X\}$, $fit_{nm_{max}}(X) = \max\{fit_{nm}(x) | x \in X\}$.

4 Elitist Strategy:

The objective of the elitist strategy is to preserve all the non-dominated chromosomes that have been obtained so far. At first, the parent chromosomes have been merged with children's chromosomes. Then duplicate and dominated chromosomes are removed. Finally, we have applied $CPKNN()$ on the newly updated population of Master GA to prune the chromosomes which reside in densely populated areas of the solution space in case the size of the Master GA exceeds the predetermined size of the same.

Algorithm 2 CPKNN(Pop_{Master} , $Size_{pop}$)

Ensure: $Size_{pop}$ is the predefined population size; Pop_{Master} is the population of Master GA.

Ensure: Pop_{Master} .

```

1:  $k = \lceil \sqrt{Size_{pop}} \rceil$ 
2:  $Pop_{Master\_KNN}[] \leftarrow$  It contains the k-th nearest neighbor of each of the chromosomes of Master GA.
3: for  $i \leftarrow 0$  to  $|Pop_{Master}|$  do
4:   Initialize  $temp\_density[]$   $\triangleright$  It contains the density estimation of chromosome  $i$  to the rest of the
     chromosome in  $Pop_{Master}$ 
5:   for  $j \leftarrow 0$  to  $|Pop_{Master}|$  do
6:      $distance_{ij} \leftarrow$  distance between  $i$ -th and  $j$ -th chromosome.
7:      $density_{ij} = \frac{1}{distance_{ij} + 2}$ 
8:      $temp\_density \leftarrow density_{ij}$ 
9:     sort  $temp\_density$  in increasing order.
10:     $Pop_{Master\_KNN}[i] = temp\_density[k]$ 
11:   end for
12: end for
13: while  $|Pop_{Master}| > Size_{pop}$  do
14:    $pos =$  position of  $\min(Pop_{Master\_KNN})$ 
15:   delete  $Pop_{Master\_KNN}[pos]$ 
16:   delete  $Pop_{Master}[pos]$ 
17: end while
18: Return  $Pop_{Master}$ 

```

4.4 Description of Algorithm 3 for QRMOCCGA

The algorithm for QRMOCCGA has been presented in algorithm 3. The algorithm 3 needs the following information as input: total number of queries in $\hat{Q}$ which has been obtained from the algorithm 1, length of the chromosome (i.e. Len_C), number of objectives (i.e. Num_{Obj}), and maximum number of iterations (i.e. Max_{itr}). The number of sub-populations has been defined by Num_{sub_pop} . It should be equal to the number of objectives. In our case, $Num_{Obj} = 2$. At the end, algorithm 3 returns Pop_{Master} which contains relevant non-dominated chromosomes.

In line 1, all the necessary variables have been initialised. $Size_{pop}$ and $Size_{sub_pop}$ are the size of the population and sub-population, respectively. Pop_{init} contains the initial population. Since Num_{sub_pop} is 2, sub_pop is a list of two lists. It contains the sub-population in every generation. $sub_pop[0]$ and $sub_pop[1]$ represent first and second sub-populations, respectively. Pop_{Master} represents population of Master GA and it will contain all the non-dominated solutions after merging the sub-populations. In other words, this is the population

of master GA. $Parent_{Slave}$ and $Parent_{Master}$ represent the parent chromosomes selected for slave GA and Master GA, respectively. Similarly, $Children_{Slave}$ and $Children_{Master}$ represent the child chromosomes for slave GA and Master GA, respectively.

In line 10, fitness values of all the chromosomes belong to the $k - th$ sub-population (i.e., $sub_pop[k]$) have been calculated according to the corresponding objectives and assigned to $Obj_{val}[k]$. In line 11, $Parent_{Slave}[k]$ has been created by applying the $selection()$ operator to select chromosomes from the $k - th$ sub-population ($sub_pop[k]$) corresponding to $k - th$ slave GA based on their values in $Obj_{val}[k]$. All the $Children_{Slave}[k]$ have been generated by applying the crossover operator on the parent chromosomes ($Parent_{Slave}[k]$) of the $k - th$ sub-population in line 12. Mutation operator has been applied on these child chromosomes $Children_{Slave}[k]$ in line 13.

In line 15, sub-populations have been merged to create the population for Master GA (i.e. Pop_{Master}). In line 16, $Parent_{Master}$ has been created by applying the $selection()$ operator to select chromosomes from the Pop_{Master} . All the $Children_{Master}$ has been generated by applying the crossover operator on the parent chromosomes $Parent_{Master}$ in line 17. Mutation operator has been applied on these child chromosomes $Children_{Master}$ in line 18. All the chromosomes belonging to $Children_{Master}$ have been merged with Pop_{Master} in line 19. An elitist strategy has been applied to Pop_{Master} in order to preserve all the non-dominated chromosomes obtained thus far, as indicated in line 20. After the completion of the predefined number of iterations (i.e. Max_{itr}), the Pop_{Master} has been returned.

Algorithm 3 QRMOCCGA($|\hat{Q}|$), $Lenc$, Num_{Obj} , Max_{itr})

Ensure: $|\hat{Q}|$ is the total number of queries in $\hat{Q}$; $Lenc$ is the length of the chromosome; Num_{Obj} indicates the number of objectives; Max_{itr} is the maximum number of iterations

Ensure: Pop_{Master} .

```

1: Initialize:  $Num_{sub\_pop} = Num_{Obj}$ ,  $Size_{pop} = \frac{|\hat{Q}|}{Lenc}$ ,  $sub\_Size_{pop} = \frac{Size_{pop}}{Num_{sub\_pop}}$   $intial\_pop = []$ ,
    $sub\_pop = [[]]$ ,  $Pop_{Master} = []$ ,  $Parent_{Slave} = []$ ,  $Children_{Slave} = []$ ,  $Parent_{Master} = []$ ,
    $Children_{Master} = []$ 
2:  $Pop_{init} = Initialize\ Population(Size_{pop}, Lenc)$ 
3: for  $i \leftarrow 0$  to  $Max_{itr} - 1$  do
4:   for  $k \leftarrow 0$  to  $Num_{Obj} - 1$  do
5:     if  $i = 0$  then
6:       Create  $sub\_pop[k]$  from  $Pop_{init}$ 
7:     else
8:       Create  $sub\_pop[k]$  from  $Pop_{Master}$ 
9:     end if
10:    Calculate  $Obj_{val}[k]$ 
11:     $Parent_{Slave}[k] = selection(sub\_pop[k], Obj_{val}[k])$ 
12:     $Children_{Slave}[k] = crossover(Parent_{Slave}[k])$ 
13:     $Children_{Slave}[k] = mutation(Children_{Slave}[k])$ 
14:  end for
15:   $Pop_{Master} \leftarrow \bigcup_{k=0}^{Num_{Obj}-1} sub\_pop[k]$ 
16:   $Parent_{Master} = selection(Pop_{Master})$ 
17:   $Children_{Master} = crossover(Parent_{Master})$ 
18:   $Children_{Master} = mutation(Children_{Master})$ 
19:   $Pop_{Master} \leftarrow Pop_{Master} \cup Children_{Master}$ 
20:   $Pop_{Master} = elitist\_strategy(Pop_{Master})$ 
21: end for
22: Return  $Pop_{Master}$ 

```

5 Experimental results

Extensive experimentation has been carried out to study the performance of our proposed QRMOCGA to solve the query recommendation problem. Experimental results have been reported in this section.

5.1 Description of the dataset

Search logs obtained from the AOL search engine [38, 44] have been used here. This data was collected from 01.03.2006 to 31.05.2006. It contains 19,442,629 search logs. Each search log looks like Fig. 1. The number of unique queries and URLs in the dataset are 4,802,520 and 1,606,326, respectively. After removing duplicate and inconsistent data, we extracted 2,671,773 number of unique query-URL pairs along with their occurrence in the dataset. Six root queries have been chosen randomly to demonstrate the effectiveness of our proposed method. These queries are- *free music, jobs, travel, pets, president, and windows*.

5.2 Experimental setup

We have generated the recommendations for the randomly chosen six queries using our proposed method, QRMOCGA. The results have been compared with the recommendations produced by a Query Recommendation system using a Single objective Genetic Algorithm (QRSGA) [6] and previously proposed two popular approaches- Heat Diffusion based method [38], SimRank based method [30], and two commercial search engines Google⁴ and Bing⁵. In QRSGA, the fitness of each chromosome has been evaluated using (15).

$$f(x) = w_1 f_1(x) + w_2 f_2(x) \quad (15)$$

where x is a chromosome, $f(x)$ represents the weighted sum of the fitness functions, $f_i(x)$ is the i_{th} objective function, and $w_i, w_i \geq 0$ is a small weight associated with $f_i(x)$.

It may be noted that the commercial search engines may return a different number of recommendations for different queries. So, only the first five recommendations for each root query have been considered here to make a fair comparison. The result has been presented in Tables 1 and 2. Since QRMOCGA generates a set of non-dominated solutions, the solution with the shortest distance value to the ideal point has been reported in the above mentioned tables.

Distance to floating ideal point: The non-dominated solutions generated by our proposed QRMOCGA are ranked based on their distance to the ideal point [42, 50]. For this purpose, the objective values of all the m non-dominated solutions are scaled in the range $[0, 1]$. Usually, the ideal point (i.e. z_{id}) has been set to $(0, 0)$ when two objective functions are both minimizations in nature regardless of the quality of the non-dominated solutions present in the Pareto front. Here we have proposed the concept of an ideal *floating* point where the

⁴ <https://www.google.com>

⁵ <https://www.bing.com/>

Table 1 Comparison between the recommended queries (free music, jobs, and travel)

QRSGA	QRMOC CGA	SimRank	Heat Diffusion	Google	Bing
Query: free music					
download free music	downloading free music	drinking irish jigs	free music downloads	free music	free music
free music download sources	free music search	irish music the bards	napster	free music download	free music downloads
free music mp3	music sharing	doctor who desktop theme	limewire	free music online	free music online
free music software	music downloads	online atv games	music downloads	free music apps	free music radio
free p2p music downloads	free oldies music	sports download games	musicmatch	free music ringtones	free music youtube
Query: jobs					
hot jobs	security jobs	www.fedjobs.com	monster	jobs near me	jobs
jobs usa	federal jobs	computer jobs	americas job bank	jobs	jobstreet
federal jobs	summer jobs	jobs in psychology	career builder	jobs hiring	jobsdb
fbi jobs	computer jobs	msnbc	monster jobs	jobs for teens	jobs hiring
summer jobs	government jobs	resumes for directv	job search	jobs indeed	jobs near me
Query: travel					
travel texas	travel guides	cheap tickets.com	sta	travelocity	travelocity
yahoo travel	cheap travel	travel.com	sta travel	travel	travelers insurance
international travel	travel agency	texas	orbitz	travelers insurance	travelocity flights
travel city	travel guide	cnn	maps	travel agency	travel
travel plans	travel deals	travel channel	expedia	travelzoo	travelzoo

values change with the Pareto front. The distance value of i_{th} solution d_i is calculated using the following equation.

$$d_i = \sqrt{\sum_{j=1}^n (z_{ij} - z_{min_j})^2} \quad (16)$$

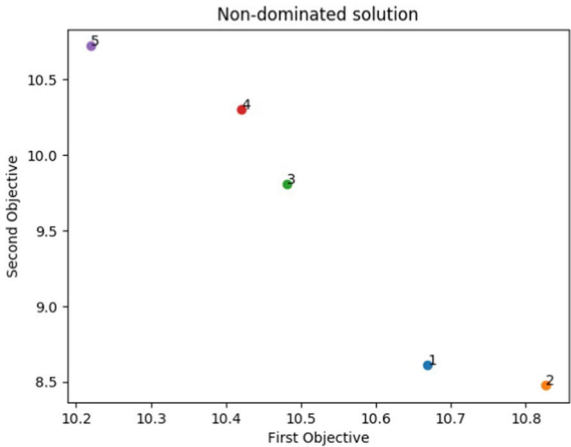
where z_{ij} is the value obtained for j_{th} objective by i_{th} solution, z_{min_j} is the minimum value for j_{th} objective (after scaling) in the Pareto front being considered, and n is the number of objective functions.

Figures 9 and 10 present the Pareto fronts generated by the proposed QRMOC CGA for the six root queries mentioned above. The solutions presented at the Pareto front for the query *president* have been reported in Table 3.

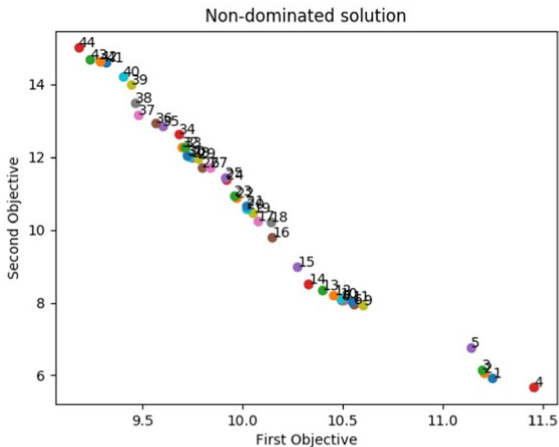
Table 2 Comparison between the recommended queries (pets, president, and windows)

QRSGA Query: pets	QRMOCGA	SimRank	Heat Diffusion	Google	Bing
lost pets	dogs	petsmart	petfinder	petsmart	petsmart
petsmart	petfinder.com	cane corso	yahoo pets	petsmart hours	petsmart locations
pets 4 you	boxer puppies	neopets	petsmart	pets	petsmart grooming
pets stores	petsmart	cobb ga lost dogs	neopets	petsmart coupons	petsmart coupons
pets for sale	pet adoption	dog breeds	avma	petsmart grooming	petsafe
Query: president					
president lincoln	abraham lincoln	civil war	george washing- ton	president trump	president trump
president reagan	presidents of the united states	wikipedia	white house	presidents	presidents
president bush	dwight eisen- hower	homework help	abraham lincoln	presidents day 2019	presidents day 2019
president bill clinton	george wash- ington	john adams	us presi- dents	president donald trump	president obama
president kennedy	calvin coolidge	www.americanpresidents.org	thomas jefferson	president of croatia	president of croatia
Query: windows					
windows updates	anderson win- dows	microsoft	microsoft	windows 10	windows 10
microsoft win- dows	microsoft win- dows	anderson win- dows	windows update	Windows	windows 10 down- load
windows explorer	windows updates	pella win- dows	anderson win- dows	Windows movie maker	windows
windows 98se	gliding win- dows	windows live	microsoft updates	Windows 10 update	windows 10 iso
arch win- dows	windows 98se	exterior doors	internet explorer	Windows 95	windows update

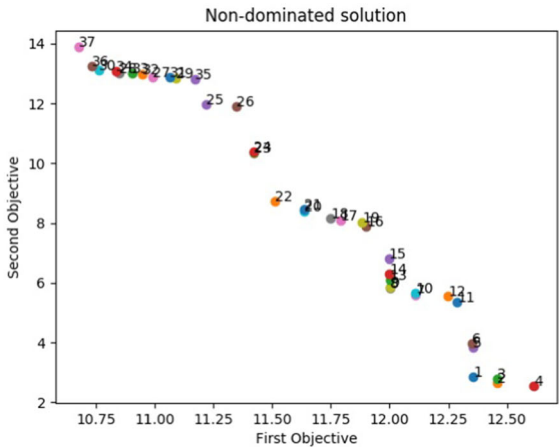
Fig. 9 Non-dominated solutions at the Pareto front generated by QRMOCCGA (Query: *free music, jobs, and travel*)



(a) free music

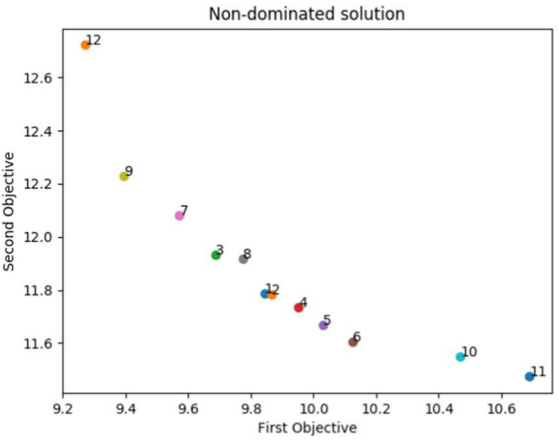


(b) jobs

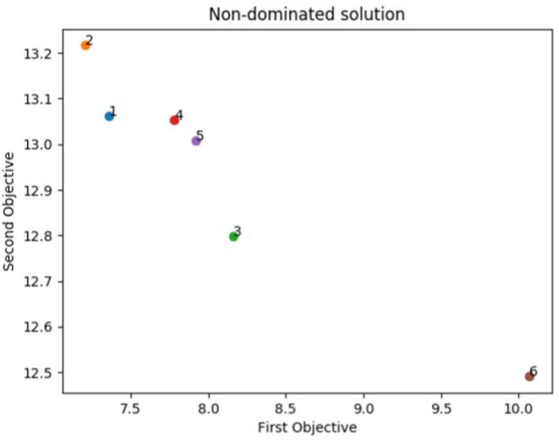


(c) travel

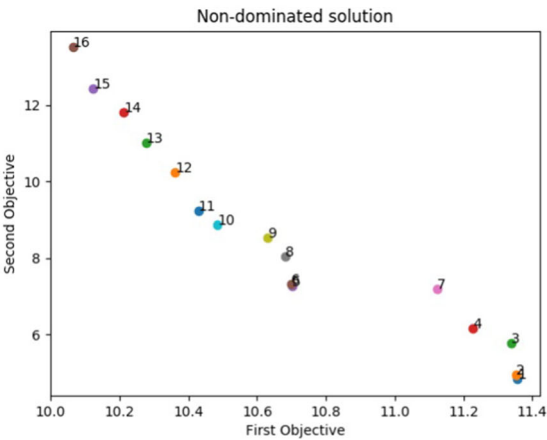
Fig. 10 Non-dominated solutions at the Pareto front generated by QRMOCGA (Query: *pets*, *president*, and *windows*)



(a) *pets*



(b) *president*



(c) *windows*

Table 3 Non-dominated solutions at the Pareto front generated by QRMOCCEA for the query *president*

Solution 1	Solution 2	Solution 3	Solution 4	Solution 5	Solution 6
abraham lincoln	abraham lincoln	presidents of the united states	abraham lincoln	marine base in tampa florida	white house design
presidents of the united states	herbert hoover	white house	condoleezza rice	presidents of the united states	presidents of the united states
dwight eisenhower	ulysses s grant	ulysses s grant	white house	herbert hoover	all the presidents of usa
george washington	john f kennedy	dwight eisenhower	herbert hoover	dwight eisenhower	dwight eisenhower
calvin coolidge	bill clinton	security	portraits of us presidents	george washington	security

5.3 Evaluation of the quality of the recommended queries

Open Directory Project (ODP) has been utilised to automatically evaluate the quality of recommended queries. It is one of the largest human-edited directories on the Web. The evaluation technique has been adopted from [5, 38]. Given a query, ODP returns a hierarchical structure or path. Each component of this path is a category. For instance, if the input query is “*Microsoft outlook*”, the output will be “*Computer: Software: Internet: Clients: Mail: Windows: Microsoft Outlook*”. Similarly, if the input query is “*Mail*”, the output will be “*Computers: Software: Internet: Servers: Mail*”. These two paths can be compared to compute the similarity between the queries “*Microsoft outlook*” and “*Mail*”. Let q_i and q_j represent two queries and the hierarchical structures or paths for these two queries are P_i and P_j , respectively. The similarity between q_i and q_j has been defined in the (17).

$$\text{sim}(q_i, q_j) = \frac{\text{lcp}(P_i, P_j)}{\max(\text{len}(P_i), \text{len}(P_j))} \quad (17)$$

where, $\text{lcp}(P_i, P_j)$ is the longest common prefix of P_i and P_j , $\text{len}(P_i)$ is the length of the path P_i . The value of $\text{sim}(q_i, q_j)$ should lie between zero and one (i.e. $0 \leq \text{sim}(q_i, q_j) \leq 1$). A sim value closer to one indicates high similarity. So, $\text{sim}(\text{microsoft outlook}, \text{mail}) = \frac{3}{7}$. Average similarity score (sim_{avg}) for the set of recommended queries are measured for the six above mentioned root queries.

5.4 Discussion

In this work, we have proposed a Query Recommendation system called QRMOCCEA to generate recommendations for a given root query. Several interesting observations have been reported in this section.

- 1 In a Cooperative Co-evolutionary evolutionary algorithm (CCEA), a problem is usually divided into several sub-problems where multiple EAs work on these sub-problems concurrently and independently. Additionally, there is a cooperation phase in which cooperation among different EAs is performed to generate the complete solution for the problem. Here

in this work, we have framed our proposed approach as CCEA. Since the query recommendation problem has been modelled as a multi-objective optimisation problem having two objectives, the entire problem has been divided into two sub-problems. Two slave GAs have been used to deal with these sub-problems. A master GA has been used to mimic the *cooperation phase*. Master GA works on the solutions provided by the Slave GAs and generates the complete solution for the entire problem.

- 2 Six different root queries have been used to generate the set of recommendations delivered by the proposed QRMOCCGA, QRSGA, SimRank [30], Heat Diffusion [38] based method, and two commercial search engines. These recommendations and the root queries have been presented in the Tables 1 and 2. The value of the weights w_1 and w_2 for QRSGA have been set to 0.5 and 0.5, respectively. It can be seen from the above mentioned two tables that all the recommendations generated by our proposed QRMOCCGA are very relevant to the original queries. Moreover, the proposed method can recommend latent semantically relevant queries. For instance, the queries *dogs*, and *boxer puppies* have been recommended for the root query *pets*. Unlike the SimRank method, the recommendations generated by the QRMOCCGA and QRSGA are clearly very relevant to the root queries.
- 3 The proposed QRMOCCGA method used Non-dominated sorting [18] to produce a set of non-dominated solutions. The Pareto front obtained for each query has been presented in Figs. 9 and 10. Generally, search engine recommends only one set of solutions in response to a given query. But the Pareto front returned by the QRMOCCGA may contain multiple non-dominated solutions. So, the solutions have been ranked according to their distance to floating ideal points and only the first ranked solution has been reported. This distance value has been defined in (16).
- 4 The number of relevant queries ($|\hat{Q}|$) obtained for the root queries *free music*, *jobs*, *travel*, *pets*, *president*, and *windows* are 23546, 4437, 7147, 1578, 613, and 1664, respectively. The stopping criterion for both QRMOCCGA and QRSGA was the maximum iteration number which has been selected depending on the number of relevant queries for a given root query.
- 5 A similarity score (*sim*) based on ODP has been defined in (17). Average similarity score (sim_{avg}) has been used to compare the performance of the proposed QRMOCCGA with the QRSGA, SimRank [30], and Heat Diffusion-[38] based method. These four methods have been deployed on the AOL dataset used in this experiment. The comparison is presented in the Fig. 11. The similarity scores obtained using the proposed QRMOCCGA method is better than those obtained by QRSGA and SimRank [30]. Although the average similarity score obtained by QRMOCCGA method is greater than that of the Heat Diffusion [38] based method for four queries, it is comparable for other two queries. Average similarity scores obtained by QRSGA for all six queries are greater than those obtained by SimRank. The average similarity score obtained by QRSGA is better than those obtained by Heat Diffusion [38] for four out of six queries. These results show the effectiveness of population-based methods to deal with query recommendation problem.
- 6 In QRMOCCGA, we have considered the number of objectives as two. Hence, at each iteration, two slave GAs execute parallelly. In the end, the solutions generated by slave GAs are merged which acts as the population for Master GA. Unlike Slave GAs, at each iteration, only a single Master GA works on the results obtained by the slave GAs. The time complexity of slave GAs for single iteration is $O(sub_size_{pop} \times Lenc)$, where sub_size_{pop} is the population size of a slave GA and $Lenc$ represents length of a chromosome. Whereas, the time complexity of Master GA is $O(size_{pop} \times Lenc + (size_{pop})^2)$. Size of the population of Master GA is $size_{pop}$. Therefore, the overall complexity of QRMOCCGA is $O(Max_{itr} \times ((sub_size_{pop} \times Lenc)) + (size_{pop} \times Lenc + (size_{pop})^2))$

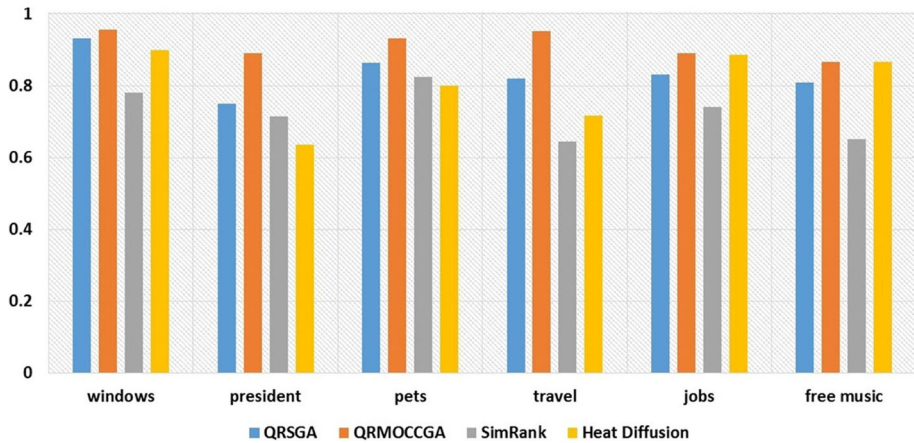


Fig. 11 Average similarity score (i.e. sim_{avg}) for the recommended queries

$\approx O(Max_{itr} \times (size_{pop})^2)$, where Max_{itr} is the maximum number of iterations. We have compared performance of the QRMOC CGA with QRSGA[6], SimRank[30], and Heat Diffusion [38] based technique. Time complexity for QRSGA is $O(Max_{itr} \times size_{pop})$. Time complexity of the SimRank is $O(Max_{itr} n^2 d_2)$ where n is the number of nodes in the graph $G_{\hat{Q}}$ and d_2 is the average of the product of the in-degrees of the possible node pairs. Time complexity of the Heat Diffusion method is $O(Max_{itr} m)$ where m is the number of edges in the graph $G_{\hat{Q}}$. Since $G_{\hat{Q}}$ is not a sparse graph, $O(Max_{itr} m) \approx O(Max_{itr} n^2)$. It may be noted that $n > size_{pop}$. So, QRMOC CGA is computationally efficient than the SimRank and Heat Diffusion method. Although QRSGA is computationally efficient than the QRMOC CGA, it has several limitations. For instance, since it is single objective based technique, it cannot generate Pareto optimal solutions. Moreover, ideal value for w_1 and w_2 is hard to determine.

6 Conclusion

A query recommendation system using Multi-Objective Cooperative Co-evolutionary Genetic Algorithm (QRMOC CGA) has been proposed in this work **to recommend a set of k number of queries for a given query**. A set of k number queries has been treated as a solution. Four features have been defined here to measure the relevancy between two queries. Based on these four features, **two objective functions have been constructed. One objective function tries to generate semantically relevant queries, whereas the other objective function tries to create queries that look similar**. In QRMOC CGA, two slave GAs independently handle these two objective functions. The solutions generated by the slave GAs have been merged and evaluated by a master GA. Since QRMOC CGA generates multiple non-dominated solutions, the system can generate various sets of recommendations for a user and the user has the freedom to choose the recommendation set according to their preference. Alternatively, a ranking system has been developed in this work to recommend the best solution to the user. Several executions of the proposed QRMOC CGA have been carried out using different queries. The number of relevant queries $|\hat{Q}|$ varies significantly for each query. In all scenarios we have found out that our proposed QRMOC CGA has been able to find out better

solution than the methods used here for comparison. In future, several other nature-inspired evolutionary algorithms can be employed to improve the quality of the recommendations. We may also introduce one or more decision variables based on semantic knowledge base and conceptual relations among queries to improve quality of the recommendation. Since our proposed QRMOCCGA returns the Pareto front, which may consists of multiple Pareto optimal solutions, we may extend it in future to generate *personalized* recommendations.

Funding No funds, grants, or other support was received

Data Availability Data available on request from the authors

Declaration

Conflicts of interest/Competing interests The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Ahmedi L, Shabani D (2017) Search engine query recommendation-using sna over query logs with user profiles. In: International Conference on Web Information Systems and Technologies, vol. 2, pp. 370–375. SCITEPRESS
2. Anagnostopoulos I, Razis G, Mylonas P, Anagnostopoulos CN (2015) Semantic query suggestion using twitter entities. *Neurocomputing* 163:137–150
3. Anagnostopoulos A, Becchetti L, Castillo C, Gionis A (2010) An optimization framework for query recommendation. In: Proceedings of the Third ACM International Conference on Web Search and Data Mining, pp. 161–170. ACM
4. Baeza-Yates R, Hurtado C, Mendoza M (2004) Query recommendation using query logs in search engines. In: International Conference on Extending Database Technology, pp. 588–596. Springer
5. Baeza-Yates R, Tiberi A (2007) Extracting semantic relations from query logs. In: Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 76–85
6. Barman D, Sarkar R, Tudu A, Chowdhury N (2020) Personalized query recommendation system: A genetic algorithm approach. *Journal of Interdisciplinary Mathematics* 23(2):523–535
7. Binshtok M, Brafman RI, Shimony SE, Martin A, Boutilier C (2007) Computing optimal subsets. In: AAAI, pp. 1231–1236
8. Bonchi F, Perego R, Silvestri F, Vahabi H, Venturini R (2012) Efficient query recommendations in the long tail via center-piece subgraphs. In: Proceedings of the 35th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 345–354. ACM
9. Cai X, Hu Z, Zhao P, Zhang W, Chen J (2020) A hybrid recommendation system with many-objective evolutionary algorithm. *Expert Syst Appl* 159:113648
10. Câmara A, Santos RL (2019) Traversing semantically annotated queries for task-oriented query recommendation. In: Proceedings of the 13th ACM Conference on Recommender Systems, pp. 511–515. ACM
11. Chai ZY, Li YL, Han YM, Zhu SF (2018) Recommendation system based on singular value decomposition and multi-objective immune optimization. *IEEE Access* 7:6060–6071
12. Cheng R, Jin Y, Olhofer M, Sendhoff B (2016) A reference vector guided evolutionary algorithm for many-objective optimization. *IEEE Trans Evol Comput* 20(5):773–791
13. Cheng H, Tan PN, Sticklen J, Punch WF (2007) Recommendation via query centered random walk on k-partite graph. In: Seventh IEEE International Conference on Data Mining (ICDM 2007), pp. 457–462. IEEE
14. Cohoon JP, Hegde SU, Martin WN, Richards D (1987) Punctuated equilibria: a parallel genetic algorithm. In: Genetic Algorithms and Their Applications: Proceedings of the Second International Conference on Genetic Algorithms: July 28–31, 1987 at the Massachusetts Institute of Technology, Cambridge, MA. Hillsdale, NJ: L. Erlbaum Associates, 1987
15. Colson B, Toint PL (2005) Optimizing partially separable functions without derivatives. *Optimization methods and software* 20(4–5):493–508

16. Cui L, Ou P, Fu X, Wen Z, Lu N (2017) A novel multi-objective evolutionary algorithm for recommendation systems. *Journal of Parallel and Distributed Computing* 103:53–63
17. Deb K, Jain H (2013) An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part i: solving problems with box constraints. *IEEE Trans Evol Comput* 18(4):577–601
18. Deb K, Pratap A, Agarwal S, Meyarivan T (2002) A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE Trans Evol Comput* 6(2):182–197
19. Deepak G, Priyadarshini JS, Babu MH (2016) A differential semantic algorithm for query relevant web page recommendation. In: 2016 IEEE International Conference on Advances in Computer Applications (ICACA), pp. 44–49. IEEE
20. Duan J, Song Y, Zhang Y, Wu M, Wang H (2018) Query recommendation using topic modeling and word embeddings. In: *Proceedings of the 2018 International Conference on Artificial Intelligence and Pattern Recognition*, pp. 86–92. ACM
21. El Ghali B, El Qadi A (2017) Context-aware query expansion method using language models and latent semantic analyses. *Knowl Inf Syst* 50(3):751–762
22. Fernández-Tobías I, Blanco R (2016) Memory-based recommendations of entities for web search users. In: *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*, pp. 35–44. ACM
23. Fonseca BM, Golgher PB, de Moura ES, Ziviani N (2003) Using association rules to discover search engines related queries. In: *Proceedings of the IEEE/LEOS 3rd International Conference on Numerical Simulation of Semiconductor Optoelectronic Devices (IEEE Cat. No. 03EX726)*, pp. 66–71. IEEE
24. Geng B, Jiao L, Gong M, Li L, Wu Y (2019) A two-step personalized location recommendation based on multi-objective immune algorithm. *Inf Sci* 475:161–181
25. Guo J, Zhu X, Lan Y, Cheng X (2017) Modeling user's search sessions for high utility query recommendation. *Information Retrieval Journal* 20(1):4–24
26. Hanauer DA, Wu DT, Yang L, Mei Q, Murkowski-Steffy KB, Vydiswaran VV, Zheng K (2017) Development and empirical user-centered evaluation of semantically-based query recommendation for an electronic health record search engine. *J Biomed Eng* 67:1–10
27. He Q, Jiang D, Liao Z, Hoi SC, Chang K, Lim EP, Li H (2009) Web query recommendation via sequential query prediction. In: 2009 IEEE 25th International Conference on Data Engineering, pp. 1443–1454. IEEE
28. Huang Z, Cautis B, Cheng R, Zheng Y, Mamoulis N, Yan J (2018) Entity-based query recommendation for long-tail queries. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 12(6):64
29. Jansen BJ, Spink A, Bateman J, Saracevic T (1998) Real life information retrieval: A study of user queries on the web. In: *ACM Sigir Forum*, vol. 32, pp. 5–17. ACM
30. Jeh G, Widom J (2002) Simrank: a measure of structural-context similarity. In: *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 538–543
31. Karabadjji NEI, Beldjoudi S, Seridi H, Aridhi S, Dhifli W (2018) Improving memory-based user collaborative filtering with evolutionary multi-objective optimization. *Expert Syst Appl* 98:153–165
32. Kumara Swamy M, Krishna Reddy P, Bhalla S (2017) Association rule based approach to improve diversity of query recommendations. In: *International Conference on Database and Expert Systems Applications*, pp. 340–350. Springer
33. Li M, Yang S, Liu X (2013) Shift-based density estimation for pareto-based algorithms in many-objective optimization. *IEEE Trans Evol Comput* 18(3):348–365
34. Li R, Li L, Wu, X, Zhou Y, Wang W (2019) Click feedback-aware query recommendation using adversarial examples. In: *The World Wide Web Conference*, pp. 2978–2984. ACM
35. Liu Y, Miao J, Zhang M, Ma S, Ru L (2011) How do users describe their information need: Query recommendation based on snippet click model. *Expert Syst Appl* 38(11):13847–13856
36. Li H, Wang Y, Zhang D, Zhang M, Chang EY (2008) Pfp: parallel fp-growth for query recommendation. In: *Proceedings of the 2008 ACM Conference on Recommender Systems*, pp. 107–114. ACM
37. Ma C, Zhang B (2018) A new query recommendation method supporting exploratory search based on search goal shift graphs. *IEEE Trans Knowl Data Eng* 30(11):2024–2036
38. Ma H, King I, Lyu MR (2011) Mining web graphs for recommendations. *IEEE Trans Knowl Data Eng* 24(6):1051–1064
39. Ma X, Li X, Zhang Q, Tang K, Liang Z, Xie W, Zhu Z (2018) A survey on cooperative co-evolutionary algorithms. *IEEE Trans Evol Comput* 23(3):421–441
40. Meunier H, Talbi EG, Reininger P (2000) A multiobjective genetic algorithm for radio network optimization. In: *Proceedings of the 2000 Congress on Evolutionary Computation. CEC00 (Cat. No. 00TH8512)*, vol. 1, pp. 317–324. IEEE

41. Ombuki B, Ross BJ, Hanshar F (2006) Multi-objective genetic algorithms for vehicle routing problem with time windows. *Appl Intell* 24(1):17–30
42. Padhye N, Deb K (2011) Multi-objective optimisation and multi-criteria decision making in sls using evolutionary approaches. *Rapid Prototyp J*
43. Paredis J (1995) Coevolutionary computation. *Artificial life* 2(4):355–375
44. Pass G, Chowdhury A, Torgeson C (2006) A picture of search. In: *Proceedings of the 1st International Conference on Scalable Information Systems*, p. 1
45. Pizzuti C (2011) A multiobjective genetic algorithm to find communities in complex networks. *IEEE Transactions on Evolutionary Computation* 16(3):418–430
46. Ponnambalam S, Ramkumar V, Jawahar N (2001) A multiobjective genetic algorithm for job shop scheduling. *Prod Plan Control* 12(8):764–774
47. Potter MA, De Jong KA (1994) A cooperative coevolutionary approach to function optimization. In: *International Conference on Parallel Problem Solving from Nature*, pp. 249–257. Springer
48. Qian C, Yu Y, Zhou ZH (2015) Subset selection by pareto optimization. In: *Advances in Neural Information Processing Systems*, pp. 1774–1782
49. Qiao D, Zhang J, Wei Q, Chen G (2017) Finding competitive keywords from query logs to enhance search engine advertising. *Inf Manag* 54(4):531–543
50. Sanchez-Gomez JM, Vega-Rodríguez MA, Perez CJ (2019) Comparison of automatic methods for reducing the pareto front to a single solution applied to multi-document text summarization. *Knowl-Based Syst* 174:123–136
51. Senthilkumar N, Ch PR (2019) Collaborative search engine for enhancing personalized user search based on domain knowledge. *J Med Syst* 43(8):243
52. Shi X, Yang CC (2007) Mining related queries from web search engine query logs using an improved association rule mining model. *J Am Soc Inf Sci Technol* 58(12):1871–1883
53. Shinde P, Waghmode R, Lokare D, Halgaonkar P (2018) Efficient query suggestion system using users search history. In: *2018 International Conference on Information, Communication, Engineering and Technology (ICICET)*, pp. 1–6. IEEE
54. Silverstein C, Marais H, Henzinger M, Moricz M (1999) Analysis of a very large web search engine query log. In: *ACM SIGIR Forum*, vol. 33, pp. 6–12. ACM
55. Song W, Liang JZ, Cao XL, Park SC (2014) An effective query recommendation approach using semantic strategies for intelligent information retrieval. *Expert Syst Appl* 41(2):366–372
56. Tekli J, Chbeir R, Traina AJ, Traina C Jr, Yetongnon K, Ibanez CR, Al Assad M, Kallas C (2018) Full-fledged semantic indexing and querying model designed for seamless integration in legacy rdbms. *Data Knowl Eng* 117:133–173
57. Tekli J, Chbeir R, Traina AJ, Traina C Jr (2019) Semindex+: A semantic indexing scheme for structured, unstructured, and partly structured data. *Knowl-Based Syst* 164:378–403
58. Vahabi H, Ackerman M, Loker D, Baeza-Yates R, Lopez-Ortiz A (2013) Orthogonal query recommendation. In: *Proceedings of the 7th ACM Conference on Recommender Systems*, pp. 33–40. ACM
59. Wang J, Huang JZ, Guo J, Lan Y (2015) Recommending high-utility search engine queries via a query-recommending model. *Neurocomputing* 167:195–208
60. Wang S, Gong M, Li H, Yang J (2016) Multi-objective optimization for long tail recommendation. *Knowl-Based Syst* 104:145–155
61. Welch WJ (1982) Algorithmic complexity: three np-hard problems in computational statistics. *J Stat Comput Simul* 15(1):17–25
62. Wen JR, Nie JY, Zhang HJ (2001) Clustering user queries of a search engine. In: *Proceedings of the 10th International Conference on World Wide Web*, pp. 162–168. Citeseer
63. Whitley D, Starkweather T (1990) Genitor ii: A distributed genetic algorithm. *Journal of Experimental & Theoretical Artificial Intelligence* 2(3):189–214
64. Wood A, Ng YK (2016) Orthogonal query recommendations for children. In: *Proceedings of the 18th International Conference on Information Integration and Web-based Applications and Services*, pp. 298–302. ACM
65. Yang Z, Tang K, Yao X (2008) Large scale evolutionary optimization using cooperative coevolution. *Inf Sci* 178(15):2985–2999
66. Yang S, Li M, Liu X, Zheng J (2013) A grid-based evolutionary algorithm for many-objective optimization. *IEEE Trans Evol Comput* 17(5):721–736
67. Zahera HM, El-Hady GF, El-Wahed WA (2013) Query recommendation for improving search engine results. In: *Information Retrieval Methods for Multidisciplinary Applications*, pp. 46–53. IGI Global
68. Zaïane OR, Strilets A (2002) Finding similar queries to satisfy searches based on query traces. In: *International Conference on Object-Oriented Information Systems*, pp. 207–216. Springer

69. Zhang Z, Nasraoui O (2006) Mining search engine query logs for query recommendation. In: Proceedings of the 15th International Conference on World Wide Web, pp. 1039–1040. ACM
70. Zhao Z, Song R, Xie X, He X, Zhuang Y (2015) Mobile query recommendation via tensor function learning. In: Twenty-Fourth International Joint Conference on Artificial Intelligence
71. Zitzler E, Laumanns M, Thiele L (2001) Spea2: Improving the strength pareto evolutionary algorithm. TIK-report 103
72. Zuo Y, Gong M, Zeng J, Ma L, Jiao L (2015) Personalized recommendation based on evolutionary multi-objective optimization [research frontier]. *IEEE Comput Intell Mag* 10(1):52–62

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.